ChatGPT

Autonomous AI Paper-Trading Agent: End-to-End Quantitative Architecture and Project Blueprint

Executive design and operating principles

The recommended system is **not an unconstrained AI that directly emits broker orders**. It should be an autonomous *decision policy operating inside a deterministic trading-control system*. The distinction is fundamental: statistical models and reinforcement learning may propose target exposures, but a separate risk governor, order-management system, reconciliation process, and kill-switch layer retain final authority over every order.

For an initial implementation, **Alpaca Paper should be the default broker integration**. Alpaca's paper environment uses the same API specification as its live environment, separate paper credentials, and simulated fills based on real-time quotes. That makes it convenient for testing an end-to-end software path. However, Alpaca explicitly states that its paper simulator does **not** model market impact, information leakage, latency-driven slippage, queue position, price improvement, regulatory fees, or dividends; it also does not constrain a paper order's quantity to displayed NBBO liquidity. Those limitations mean broker-reported paper P&L must not be treated as an unbiased forecast of live performance. [1]

Interactive Brokers should be implemented later as a second `BrokerAdapter`. IBKR currently exposes REST/WebSocket Web APIs as well as its asynchronous TCP-based TWS API, which is attractive if the project later needs a broader instrument or order-type surface. IBKR also provides simulated/risk-free testing facilities; its public free-trial environment currently uses market data delayed by 10–15 minutes for most products, so that particular environment is not a substitute for a real-time intraday evaluation feed. [2]

The core engineering decisions should therefore be:

Concern	Recommended decision
Trading mode	Paper-only credentials and endpoints; no live credentials present in the runtime
First universe	Highly liquid U.S. equities/ETFs, minute bars; expand only after correctness is demonstrated
Decision hierarchy	Econometric/rule baselines first; PPO added as a challenger
RL action	**Target portfolio exposure**, not raw order messages
Regime model	HMM producing filtered regime probabilities
Mean reversion	Cointegration-aware OU spread model
Stochastic model	Brownian/GBM approximation for scenario/risk calculations, augmented by empirical stresses

Concern	Recommended decision
Position sizing	Volatility targeting as primary scaler; optional fractional Kelly overlay
Safety	Independent deterministic risk governor that can reject any model action
Research backtester	vectorbt for high-throughput experiments
Final backtester	Event-driven simulator sharing production trading components
Market-data persistence	Immutable raw data plus cleaned canonical Parquet datasets
Runtime transport	`asyncio` queues initially; introduce Kafka/Redpanda only when scale justifies it
Broker	Alpaca Paper first; IBKR adapter second
ML	PyTorch + Stable-Baselines3 PPO
Observability	Prometheus/Grafana for operational/risk metrics, W&B/MLflow for experiments, Streamlit for a research/operator view
Deployment	One Dockerized host initially; avoid Kubernetes until multiple independently scaled services require it
Promotion model	Explicit champion/challenger process; **never automatic paper-to-live promotion**

Stable-Baselines3 is a PyTorch-based RL implementation suite with consistent algorithm interfaces, and its PPO implementation is a practical choice for a bounded continuous target-allocation action space. PPO was designed around repeated minibatch optimization of a surrogate policy objective while limiting excessively large policy updates; its original work emphasized the favorable practical balance between implementation complexity and empirical sample efficiency. [3]

For quantitative research, vectorbt is particularly useful at the *discovery* stage because its architecture operates over pandas/NumPy structures and is designed for large parameter and strategy sweeps. It should not, however, be the only validation engine: the final pre-paper test needs to reproduce order lifecycle, partial fills, event timing, latency, and broker-state reconciliation. [4]

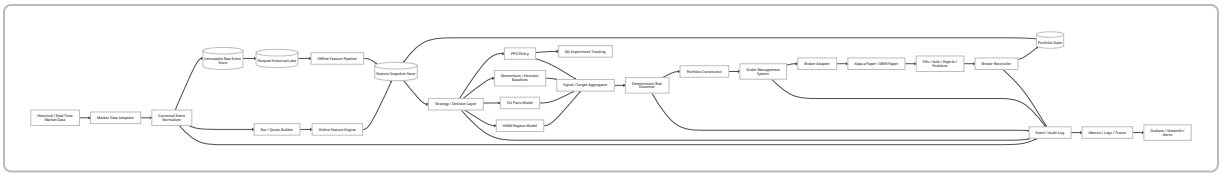
A useful project-level objective is:

> **Build a deterministic, replayable, auditable trading platform in which HMM/OU/rule/RL strategies are interchangeable decision modules, while data integrity, sizing, risk, execution, and broker reconciliation remain strategy-independent.**

That design prevents the most dangerous architectural mistake in autonomous trading: putting data access, alpha logic, sizing, risk controls, and order submission inside one opaque "agent."

System architecture and data flow

Target architecture



The real-time market-data adapter should be a **single centralized connection that fans data out internally**, rather than allowing each strategy to open a WebSocket independently. Alpaca's current documentation says many subscriptions allow only one active WebSocket connection, and slow clients can be disconnected if their outgoing buffer fills. That strongly favors a single ingest process with bounded internal queues, backpressure monitoring, and durable replay. [5]

For deeper tick/order-book work, a vendor such as Databento can complement broker data. Its current interfaces expose both historical and live data, multiple asset classes, tick-to-minute frequencies, order-book schemas, and examples for execution-slippage and queue-position analysis. Its timestamp model explicitly distinguishes event, receive, and send timestamps; that is exactly the distinction a serious event-driven simulator needs. [6]

Canonical event contracts

Do not pass raw vendor messages into models. Normalize every source into internal versioned schemas such as:

```
MarketEvent(
    event_id,
    source,
    symbol,
    venue,
    event_type,
    event_time,       # exchange / market event timestamp
    receive_time,     # local/vendor receipt timestamp
    price,
    size,
    bid,
    ask,
    bid_size,
    ask_size,
    sequence_number,
    schema_version
)

FeatureSnapshot(
```

```
    symbol,
    asof_time,
    feature_version,
    values,
    source_watermark,
    is_stale
)

Signal(
    strategy_id,
    model_version,
    asof_time,
    symbol,
    raw_score,
    confidence,
    target_weight,
    reason_codes
)

OrderIntent(
    intent_id,
    strategy_id,
    symbol,
    target_quantity,
    max_slippage_bps,
    urgency,
    expires_at
)

RiskDecision(
    intent_id,
    approved,
    resized_quantity,
    rule_hits,
    portfolio_snapshot_id
)
```

Every computed object should carry an `asof_time`. The policy must be prohibited from consuming information whose event time or publication time is later than the simulated/live decision time. This single convention eliminates a large class of accidental look-ahead bugs.

Storage hierarchy

Use three logical levels:

Layer	Contents	Mutation policy	Suggested implementation
Raw	Vendor messages, trades, quotes, bars, reference data	Append-only	Compressed Parquet/object storage
Canonical	Adjusted and normalized bars/quotes/events	Rebuildable	Parquet partitioned by dataset/date/symbol
Features	Versioned feature snapshots	Rebuildable from canonical data	Parquet/DuckDB offline; memory/Redis/Postgres for runtime state

For minute-bar work, this is substantially simpler than deploying a full enterprise feature-store platform on day one. Introduce Feast or a comparable system only after multiple independent training/inference workloads make feature lineage difficult to manage.

A particularly important data-engineering rule is to preserve **both market event time and ingestion/receive time**. Market-data systems can expose several timestamp semantics, and even exchange-originated event timestamps can contain ordering complications across gateways. Databento's documentation, for example, distinguishes `ts_event`, publisher-send timing, receive time, and live send time, and notes that source timestamps are preserved rather than silently corrected. [7]

Repository structure

A practical monorepo is:

```
ai-paper-trader/
├── configs/
│   ├── strategies/
│   ├── risk/
│   ├── brokers/
│   └── environments/
├── data_contracts/
├── notebooks/
├── src/
│   ├── data/
│   │   ├── adapters/
│   │   ├── normalization/
│   │   ├── quality/
│   │   └── replay/
│   ├── features/
│   ├── models/
│   │   ├── hmm/
│   │   ├── ou/
│   │   ├── stochastic_risk/
│   │   └── ppo/
│   ├── strategies/
```

```
│   ├── portfolio/
│   ├── risk/
│   ├── execution/
│   │   ├── oms/
│   │   ├── alpaca/
│   │   └── ibkr/
│   ├── backtest/
│   ├── observability/
│   └── common/
├── tests/
│   ├── unit/
│   ├── property/
│   ├── integration/
│   ├── replay/
│   └── fault_injection/
├── docker/
├── migrations/
└── pyproject.toml
```

Research notebooks should import production library code; production code should **never import notebooks**. All experiment-defining values—universe, dates, thresholds, costs, model parameters, seeds, feature versions—belong in immutable run configurations.

Quantitative models and autonomous decision engine

Hidden Markov regime model

Treat market regime as a latent state:

$$Z_t \in \{1, \dots, K\},$$

with transition matrix

$$A_{ij} = P(Z_t = j \mid Z_{t-1} = i)$$

and observation vector

$$Y_t = \left[r_t,\ \hat{\sigma}_t,\ \text{range}_t,\ \text{volume surprise}_t,\ \text{breadth}_t \right].$$

For a Gaussian HMM,

$$Y_t \mid Z_t = k \sim \mathcal{N}(\mu_k, \Sigma_k).$$

HMM/regime-switching approaches have been used in financial applications precisely to infer unobserved market states from observable return/risk variables; more recent research continues to explore combining inferred regimes with portfolio decision layers. This should be treated as a state-estimation mechanism, not evidence that HMM regime labels themselves constitute guaranteed alpha. [8]

Critical implementation rule: live features use the *filtered* probability

$$p_t(k) = P(Z_t = k \mid Y_1, \ldots, Y_t),$$

not a full-sample smoothed probability

$$P(Z_t = k \mid Y_1, \ldots, Y_T), \quad T > t.$$

Using future observations to retrospectively smooth the regime and then feeding that regime into a historical trading rule is look-ahead leakage.

Start with $K = 2$–4 candidate regimes and select based on out-of-sample stability rather than merely maximum in-sample likelihood. A useful interpretation layer might map states post-fit to:

Empirical state	Possible characterization	Portfolio behavior
Low-volatility / positive drift	Calm trend	Momentum strategies permitted
Low-volatility / low drift	Range/mean-reversion	Pairs allocation permitted
High-volatility / negative drift	Risk-off	Reduce gross exposure
High-volatility / rebound	Transitional	Low confidence; tighten caps

The strategy should consume the entire posterior vector p_t, not just an unstable hard label. Also expose HMM entropy,

$$H_t = -\sum_k p_t(k) \log p_t(k),$$

so ambiguous regimes automatically reduce conviction.

Ornstein-Uhlenbeck pairs model

For a candidate pair A, B, construct a spread such as

$$X_t = \log P_t^A - \beta \log P_t^B,$$

where β is estimated from an appropriate historical window. Candidate selection should first establish an economically sensible relationship and test whether the spread behaves sufficiently stationarily; do not search thousands of unrelated pairs and simply retain whichever produces the prettiest backtest.

Model the spread as:

$$dX_t = \kappa(\mu - X_t)\, dt + \sigma\, dW_t,$$

where:

- μ is long-run equilibrium,

- $\kappa > 0$ is speed of mean reversion,
- σ is diffusion volatility.

OU-based portfolio fitting and parameter estimation are well-established approaches in mean-reversion research, but empirical work also illustrates the danger of assuming OU dynamics remain stationary indefinitely. That makes model invalidation and rolling re-estimation part of the strategy itself, rather than an optional refinement. [9]

The exact discretization over interval Δ is

$$X_{t+\Delta} = \mu + (X_t - \mu)e^{-\kappa\Delta} + \sigma\sqrt{\frac{1 - e^{-2\kappa\Delta}}{2\kappa}}\epsilon_t, \qquad \epsilon_t \sim N(0,1).$$

The stationary variance is

$$\mathrm{Var}(X) = \frac{\sigma^2}{2\kappa}$$

and the model half-life is

$$t_{1/2} = \frac{\ln 2}{\kappa}.$$

Define:

$$z_t = \frac{X_t - \mu}{\sigma/\sqrt{2\kappa}}.$$

A transparent baseline policy is then:

```
z > +z_entry -> short spread
z < -z_entry -> long spread
|z| < z_exit -> close
|z| > z_stop -> emergency close
holding_time > max_holding -> close
stationarity/model test fails -> disable pair
HMM risk regime detected -> block new entry / reduce sizing
```

The entry, exit, and stop thresholds should be chosen on nested training/validation data, not tuned against the final test sample. Mean-reversion research has explicitly treated entry, exit, profit-taking, stop-out, and finite holding horizons as parts of the same stopping problem. [10]

Stochastic calculus and Brownian risk layer

For a liquid equity over sufficiently short horizons, start with the geometric Brownian approximation

$$dS_t = \mu S_t\,dt + \sigma S_t\,dW_t.$$

It implies

$$S_{t+\Delta} = S_t \exp\left[\left(\mu - \frac{1}{2}\sigma^2\right)\Delta + \sigma\sqrt{\Delta}Z\right], \quad Z \sim N(0,1).$$

For a smooth portfolio value $V(S,t)$, Itô's lemma yields

$$dV = \left(V_t + \mu S V_S + \frac{1}{2}\sigma^2 S^2 V_{SS}\right)dt + \sigma S V_S \, dW_t.$$

In this system, the Brownian model should be used primarily for **risk approximation and scenario generation**, not as a literal belief that intraday returns are perfectly Gaussian. Complement diffusion scenarios with:

- block-bootstrap historical returns;
- volatility multipliers;
- instantaneous gap shocks;
- correlation-to-one stresses;
- spread widening;
- delayed execution;
- pair-decoupling shocks.

This avoids relying on the diffusion model precisely when markets depart most sharply from its assumptions.

For portfolios containing options, the same risk layer can eventually produce scenario Greeks and implied-volatility shocks. A richer market-data source can supply option chains, implied volatility, and related fields when that capability is added. [11]

PPO decision layer

PPO should be introduced **after** HMM, OU, momentum, and simple allocation baselines work.

Recommended observation:

$$s_t = [\ \underbrace{r_{t-k:t}}_{\text{return history}},\ \underbrace{\hat{\sigma}_t}_{\text{volatility}},\ \underbrace{p_t(Z)}_{\text{HMM posterior}},\ \underbrace{z_t^{OU}}_{\text{spread state}},\ \underbrace{m_t}_{\text{momentum}},\ \underbrace{q_t}_{\text{liquidity}},\ \underbrace{w_t}_{\text{current positions}},\ \underbrace{c_t}_{\text{cash}},\ \underbrace{D_t}_{\text{drawdown}},\ \underbrace{\tau_t}_{\text{time}},\ \underbrace{f_t}_{\text{system flags}}\ ].$$

Stable-Baselines3 supports dictionary/multiple-input observations through its policy interfaces, so heterogeneous market, portfolio, and control state can remain logically separated before encoding. [12]

Recommended action:

$$a_t = \tilde{w}_{t+1},$$

where $\tilde{w}_{t+1}$ is a vector of **desired portfolio weights**. The agent does *not* choose `BUY 1,000 AAPL MARKET`; a portfolio constructor compares desired and current exposure, the risk governor clips or rejects the proposal, and the OMS decides how to implement the residual trade.

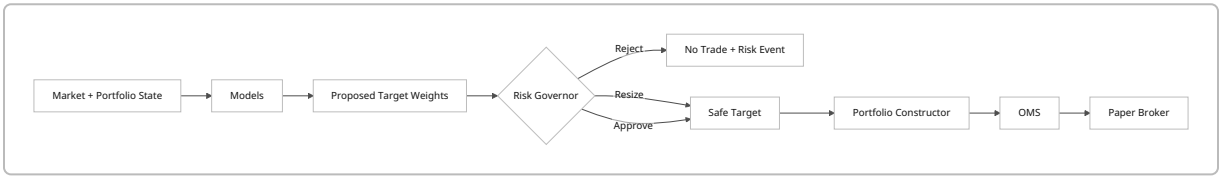
A useful training reward is:

$$R_t = \Delta \mathrm{NAV}_t - \lambda_{tc} C_t - \lambda_{turn} T_t - \lambda_{dd} \phi(D_t) - \lambda_{risk} \psi(\sigma_{p,t}),$$

with transaction costs C_t, turnover T_t, drawdown penalty ϕ, and risk penalty ψ.

Do not encode hard risk limits solely as reward penalties. Work on constrained RL explicitly distinguishes optimization objectives from safety constraints; here, the simpler and safer design is to enforce leverage, concentration, drawdown, stale-data, and order-rate limits in a deterministic external governor whether or not the PPO policy "learns" to respect them. [13]

The production hierarchy should consequently be:



Strategy arbitration

Do not make PPO responsible for discovering every behavior simultaneously. A more interpretable initial ensemble is:

$$w_t^{raw} = \alpha_{OU,t} w_{OU,t} + \alpha_{MOM,t} w_{MOM,t} + \alpha_{RL,t} w_{RL,t},$$

where coefficients may depend on HMM state probabilities:

$$\alpha_{j,t} = g_j(p_t(Z), \hat{\sigma}_t, \mathrm{liquidity}_t).$$

Initially set $\alpha_{RL} = 0$. Promote PPO first to **shadow mode**, then to a capped small share of the paper portfolio only after it adds value over deterministic baselines across multiple untouched walk-forward folds.

Backtesting, overfitting control, and portfolio risk

Two-level backtesting

Use **vectorbt for breadth** and an **event-driven simulator for truth-testing**.

Vectorbt is designed to make large parameter/strategy experiments computationally efficient over pandas/NumPy-style objects, which makes it suitable for discovering broad regions of stable parameters rather than a single optimized point. [14]

The event-driven simulator should reuse as much production code as possible:

```
HistoricalEventReplay
        ↓
```

```
same FeatureEngine
        ↓
same Strategy
        ↓
same RiskGovernor
        ↓
same PortfolioConstructor
        ↓
same OMS
        ↓
SimulatedBrokerAdapter
        ↓
same FillReconciler
```

That eliminates the common "two strategy implementations" failure mode where the research notebook and paper-trading system subtly behave differently.

Walk-forward validation

For each fold:

```
TRAIN                EMBARGO       VALIDATE       TEST
|-------------------|....|---------|-------------|
      fit all                    choose           frozen
      parameters                 candidate        evaluation
```

A representative procedure is:

```
for fold in chronological_folds:
    train = data[fold.train]
    validation = data[fold.validation]
    test = data[fold.test]

    # Fit ONLY on training history
    scaler.fit(train)
    hmm.fit(train)
    ou_models.fit(train)
    ppo.fit(train_environment)

    # Select hyperparameters/model candidate
    scores = evaluate_on(validation)

    # Freeze every parameter
    champion = select(scores).freeze()
```

```
    # One untouched evaluation
    report = evaluate_on(test, champion)
```

Anything learned from data—normalization constants, HMM state parameters, hedge ratios, feature-selection thresholds, transaction-cost calibration, RL observation statistics—must be fitted inside the fold.

If prediction labels or trades overlap across time, use purging/embargo logic so training observations cannot leak information from test-period outcomes.

Backtest-overfitting controls

The research process itself is a source of selection bias. The Deflated Sharpe Ratio was developed specifically to adjust performance assessment for multiple testing/selection and non-normal return distributions, while the Probability of Backtest Overfitting framework addresses the risk of selecting a strategy that excels in-sample but deteriorates out of sample. [15]

Therefore every experiment registry should record **failed runs as well as successful ones**. Otherwise the project cannot tell whether a Sharpe ratio came from one hypothesis or from the winner of thousands of silent trials.

A candidate should be reported with at least:

Dimension	Required evaluation
Return	CAGR/annualized return, average daily return
Risk-adjusted	Sharpe, Sortino, Deflated Sharpe
Tail risk	Max drawdown, expected shortfall, worst day/week
Trading	Turnover, trades/day, holding time, win/loss distribution
Execution	Spread cost, modeled slippage, fill ratio
Stability	Fold-by-fold results, regime results, parameter sensitivity
Capacity proxy	Pct. of displayed volume / ADV, spread sensitivity
Selection bias	Number of attempted variants; PBO/DSR diagnostics
Robustness	Delayed signal, doubled cost, widened spread, missing-data tests

Reject a model that works only at a single exact threshold. Prefer broad performance plateaus.

Transaction-cost simulation

Each simulated execution should decompose:

$$C_{trade} = C_{spread} + C_{slippage} + C_{impact} + C_{fees} + C_{borrow} + C_{latency}.$$

For paper trading, maintain **two P&L books**:

1. **Broker paper P&L**: whatever the paper broker reports.
2. **Conservative shadow P&L**: reprice fills using the internal cost/latency model.

This is especially important with Alpaca because its documentation explicitly says the paper environment omits market impact, latency slippage, order queue position, regulatory fees, dividends, and—currently—borrow fees, and can paper-fill quantities exceeding the available displayed NBBO size. [1]

A shadow fill model could use:

$$P_{\text{buy}} = P_{\text{ask}} + \eta_1 \,\text{spread} + \eta_2 \sigma \sqrt{\Delta t} + \eta_3 \sqrt{\frac{Q}{ADV}} P$$

and analogous adverse pricing for sells. The coefficients are deliberately conservative calibration parameters, not universal constants.

Dynamic position sizing

Volatility targeting

Use volatility targeting as the most transparent primary allocator:

$$s_t = \min\left(L_{\max}, \frac{\sigma_{\text{target}}}{\max(\hat{\sigma}_{p,t}, \epsilon)}\right)$$

and

$$w_t = s_t w_t^{raw}.$$

Smooth $\hat{\sigma}_{p,t}$ to prevent position oscillation and add explicit turnover limits.

Kelly overlay

Kelly's original growth-optimal framework maximizes expected logarithmic wealth growth. [16]

For a simplified multivariate return approximation,

$$w_{\text{Kelly}} \approx \Sigma^{-1} \mu.$$

In practice, μ is extremely estimation-sensitive, so this architecture should treat Kelly as a **capped sizing input**, not the sole allocator:

$$w_t^{\star} = \alpha w_{\text{Kelly}}, \qquad 0 < \alpha < 1.$$

A fractional setting such as $\alpha = 0.10$–0.25 is a reasonable *engineering experiment range*, not a theoretically universal optimum. Final exposure remains constrained by volatility targets, concentration limits, and the risk governor.

Deterministic risk governor

The risk governor should run twice: once when a target is proposed and again immediately before an order is submitted.

Illustrative initial **paper-testing limits** are:

Control	Illustrative policy	Action
Stale 1-minute input	> 2 bar intervals	Block new risk
Single-name weight	5% NAV	Clip
Pair gross exposure	10% NAV	Clip
Portfolio gross	100% NAV initially	Clip
Portfolio net	±30% NAV	Clip
Strategy daily loss	1% warning	Disable new entries
Portfolio daily loss	2%	Cancel opens and halt
Drawdown	5% warning / 8% halt	De-risk / kill switch
Order rejects	3 in 60 seconds	Halt execution
Broker/feed heartbeat loss	Configured threshold	Cancel opens; safe mode
Unexpected position	Any material mismatch	Halt and reconcile
Extreme slippage	> rolling tolerance	Disable affected strategy
Model unavailable	Any required component	Fallback/no-trade

These are starting values for software testing, not recommended investment limits. All thresholds should be configuration-controlled, unit-tested, and chosen for the specific universe and horizon.

For an OU position, add model-specific exits:

$$|z_t| > z_{\text{stop}},$$

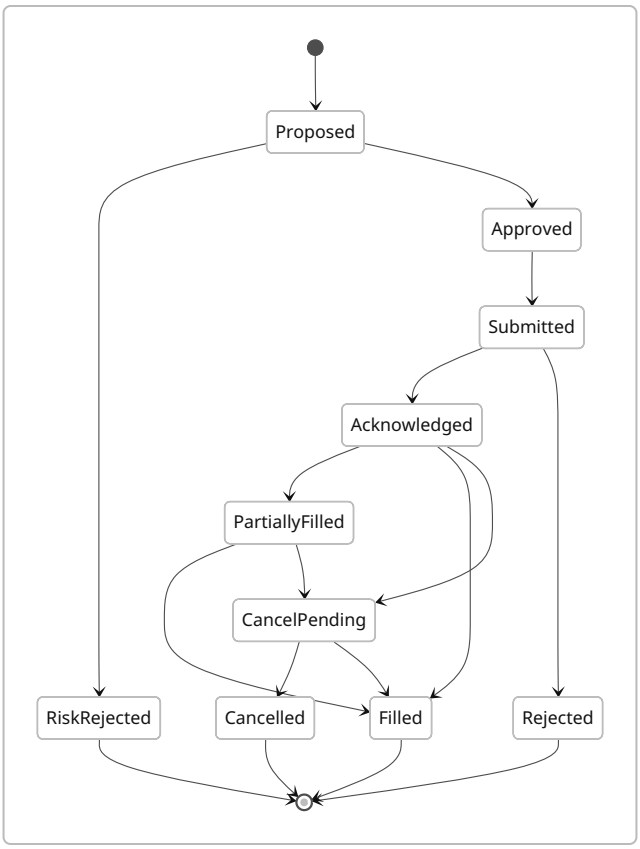
loss of stationarity/model validity, a structural regime change, or holding time beyond a configurable multiple of estimated half-life.

The portfolio-level kill switch is intentionally independent of the strategy. A strategy cannot override it.

Execution, reliability, and observability

Order-management state machine

Orders need an explicit lifecycle:



Never infer that a REST request timeout means an order failed. After ambiguous network outcomes, reconcile with the broker using an idempotent `client_order_id` before submitting anything again.

The execution service should enforce:

- globally unique and idempotent order IDs;
- persistent order state;
- duplicate-fill detection;
- out-of-order event handling;
- partial-fill accounting;
- retry with bounded exponential backoff;
- broker rate-limit handling;
- open-order reconciliation after restart;
- account/position reconciliation before resuming trading;
- deadline/expiry semantics for stale order intents.

Alpaca itself warns that live-style algorithm testing must deal with nonfills, price spikes, disconnections, and retries even though the paper order is ultimately simulated, which makes these software-failure paths essential paper-trading tests. [1]

Broker abstraction

Use a narrow interface:

```python
class BrokerAdapter(Protocol):
    async def get_account(self) -> Account: ...
    async def get_positions(self) -> list[Position]: ...
    async def get_open_orders(self) -> list[Order]: ...
    async def submit_order(self, order: BrokerOrder) -> BrokerOrderId: ...
    async def cancel_order(self, order_id: BrokerOrderId) -> None: ...
    async def replace_order(self, ...) -> None: ...
    async def stream_order_events(self) -> AsyncIterator[OrderEvent]: ...
```

Nothing above this boundary should reference Alpaca-specific or IBKR-specific JSON structures.

Alpaca's current paper setup explicitly uses separate paper credentials while preserving the same API specification between paper and live environments. That is useful for adapter testing, but the project should still prevent a live base URL from being accepted by a paper deployment. [1]

Use an explicit environment guard such as:

```python
assert settings.trading_mode == "PAPER"
assert broker.account_type == "PAPER"
assert "paper" in broker.endpoint_host
assert not secrets.contains_live_credentials()
```

Data-pipeline failure semantics

A trading system should become **less active as uncertainty about system state increases**.

Examples:

```
Market-data gap
    -> mark features stale
    -> block new positions
    -> preserve existing risk exits

Feature service crash
    -> no model action
    -> risk engine remains alive

Model crash
    -> fallback baseline or flat/no-trade
    -> never "use last prediction forever"

Broker stream disconnect
    -> suspend submissions
    -> REST reconcile
```

```
    -> restart stream
    -> compare orders/positions
    -> resume only if state agrees

Unknown position
    -> HALTED
    -> operator-visible reconciliation incident
```

The WebSocket reader must not perform model inference synchronously. Alpaca notes that slow consumers can be disconnected; therefore the reader should deserialize, validate, timestamp, enqueue, and return immediately, while downstream workers consume from bounded queues. [17]

Observability model

Grafana's alerting stack is designed to evaluate metrics/log-derived conditions and generate alerts, making it appropriate for the infrastructure and risk-control side of this platform. [18]

Create separate dashboards for:

Domain	High-value metrics
Portfolio	NAV, realized/unrealized P&L, gross/net exposure, leverage, drawdown
Strategy	P&L by strategy, turnover, hit rate, position count, signal strength
HMM	Regime probabilities, regime entropy, transitions, parameter drift
OU	Z-score, κ, half-life, spread variance, model-valid flag
PPO	Episode reward, action distribution, action saturation, entropy, approximate KL, value loss
Execution	Submit-to-ack, ack-to-fill, fill ratio, rejects, cancels, partial fills, shadow slippage
Data	Event lag, bar latency, gaps, duplicates, stale-symbol count
System	CPU/RAM, queue depths, WebSocket reconnects, API errors, heartbeat
Governance	Model version, feature version, config hash, deployment SHA

Every decision needs a reconstructable audit record:

```
{
  "decision_id": "...",
  "timestamp": "...",
  "feature_version": "...",
  "model_version": "...",
  "portfolio_before": "...",
  "regime_probabilities": [0.12, 0.78, 0.10],
```

```
  "raw_model_target": {"A": 0.08, "B": -0.07},
  "risk_adjusted_target": {"A": 0.05, "B": -0.05},
  "risk_rules_triggered": ["PAIR_GROSS_CLIP"],
  "orders_created": ["..."],
  "config_hash": "...",
  "git_sha": "..."
}
```

This enables the most important debugging question in autonomous trading:

> **Why, using exactly the information available at that instant, did the system place this order?**

Technical stack

Layer	Recommended stack	Role
Language	Python	Research and runtime
Tabular computation	Pandas, NumPy; optionally Polars	Feature/research pipelines
Statistics	SciPy, statsmodels	Tests, estimation, optimization
HMM	hmmlearn/custom PyTorch implementation	Regime inference
ML	PyTorch	Neural components
RL	Stable-Baselines3 + Gymnasium	PPO training/environment
Fast research backtesting	vectorbt	Parameter sweeps and signal tests
Production-style backtesting	Custom event simulator	Order/fill/reconciliation fidelity
Serialization	Pydantic + Arrow/Parquet	Typed contracts and historical storage
Analytical query	DuckDB	Local research over Parquet
Metadata/state	PostgreSQL	Runs, positions, orders, model registry metadata
Cache/runtime	Redis optional	Latest features/state
Streaming	`asyncio` first; Kafka/Redpanda later	Event distribution
Market data	Broker feed + optional Databento	Minute/tick/order-book data
Paper broker	Alpaca first, IBKR second	Execution simulation

Layer	Recommended stack	Role
Metrics	Prometheus	Time-series operational metrics
Dashboards	Grafana + Streamlit	Operations and research
ML tracking	W&B or MLflow	Runs/models/artifacts
Packaging	`pyproject.toml` + locked environment	Reproducibility
Containers	Docker/Compose	Deployment isolation
CI	pytest, mypy/pyright, Ruff, property tests	Quality gates

Databento's current platform supports historical/live feeds, OHLCV through order-book representations, and multiple sampling frequencies, so it is a credible upgrade when the project moves beyond broker-provided minute bars into execution/microstructure experiments. [19]

Sequential implementation roadmap

The following is a five-phase plan. The suggested durations assume a focused one-to-two-engineer implementation and are planning estimates rather than fixed deadlines.

Research & Data

Suggested duration: two to three weeks.

The purpose of this phase is not to maximize strategy returns. It is to prove that data can be obtained, normalized, replayed, and reproduced without leakage.

Work packages

Workstream	Action
Scope	Fix initial universe, session hours, bar interval, capital, and allowed asset types
Historical data	Acquire adjusted and unadjusted minute history plus corporate-action/reference data
Live data	Build WebSocket market-data adapter
Canonical schema	Implement `MarketEvent`, `Bar`, `FeatureSnapshot` contracts
Data QC	Detect gaps, duplicate bars, impossible OHLC, zero volume, crossed quotes, timezone errors
Storage	Raw immutable + normalized Parquet
Replay	Deterministic historical event replay

Workstream	Action
Benchmark	Buy/hold, cash, simple momentum, naive mean-reversion baselines
Experiment registry	Store config, seed, git SHA, data hash, outputs

For Alpaca users, note that a paper-only account is currently entitled to IEX data, while the paper environment itself uses real-time quote-driven simulated fills. Data-source alignment between historical research and the paper feed must therefore be documented explicitly. [1]

Deliverables

- data specification;
- data-quality report;
- reproducible historical dataset manifest;
- WebSocket capture service;
- event replayer;
- baseline vectorbt notebook/package;
- initial metrics dashboard;
- data-lineage documentation.

Exit criteria

Two independent runs against the same immutable dataset and configuration produce identical feature values and strategy output; no future information appears in any feature; missing-data behavior is defined and tested.

Math Model Prototyping

Suggested duration: three to four weeks.

Build each quantitative component independently before combining them.

HMM deliverables

Implement:

$$Y_t \to P(Z_t = k \mid Y_{1:t})$$

with rolling walk-forward fits, regime posterior histories, transition matrices, entropy, and regime-duration statistics.

Test:

- different K;
- different observation sets;
- transition stability;

- sensitivity across train windows;
- performance with a one-bar execution delay.

OU deliverables

Implement:

$$X_t = \log A_t - \beta \log B_t,$$

OU parameter estimation, half-life, Z-score, entry/exit/time-stop logic, rolling validity tests, and pair invalidation.

Research on OU trading supports fitting model parameters directly to mean-reverting portfolios, while other empirical studies underscore that nonstationarity can erase apparent advantages; that tension should be made explicit in the model card. [9]

Stochastic-risk deliverables

Build:

- GBM simulation;
- realized/EWMA volatility;
- covariance estimator;
- Monte Carlo loss distribution;
- historical block bootstrap;
- gap and volatility shocks;
- expected-shortfall reporting.

Baseline signal stack

At the end of this phase there should be at least:

```
HMM regime gate
        +
OU pair signal
        +
simple momentum benchmark
        +
volatility-targeted sizing
        +
transaction-cost model
```

Exit criteria

Every mathematical output has a unit test against hand-calculated or synthetic data; no model is selected exclusively on the final holdout; performance remains qualitatively plausible under doubled trading costs and a one-bar delay.

Agent Core Development

Suggested duration: four to five weeks.

This phase creates the autonomous policy layer and production control plane.

Build a Gymnasium-compatible environment whose `step()` lifecycle is conceptually:

```
observation_t
    -> policy action / target weights
    -> deterministic risk adjustment
    -> simulated orders
    -> fill model
    -> portfolio update
    -> next market event
    -> reward
```

Start PPO only after the deterministic strategies exist. Stable-Baselines3 is appropriate for the implementation, while the original PPO work provides the underlying policy-gradient design. [20]

Required components

Component	Deliverable
RL environment	Time-consistent state/action/reward interface
PPO	Seeded reproducible training pipeline
Strategy protocol	Uniform `predict_target()` interface
Ensemble	Rules for combining HMM, OU, momentum, PPO
Risk governor	Stateless rules plus portfolio-state rules
Portfolio constructor	Translate safe weights into quantities
OMS	Durable order-state machine
Replay engine	Same interfaces as runtime
Model registry	Feature/model/config version relationships
Shadow mode	Evaluate PPO without giving it order authority

Run PPO against three comparators: cash, deterministic strategy ensemble, and a randomized or trivial policy. An RL agent that merely reconstructs a basic momentum rule at much greater complexity should not be promoted.

Exit criteria

The same frozen strategy produces materially identical decisions in backtest replay and dry-run replay; all hard-risk rules have property tests demonstrating that arbitrary agent output cannot violate them.

Paper Trading Integration

Suggested duration: three to four weeks plus a soak period.

Implement Alpaca first. Alpaca's paper environment is a real-time simulation, uses a separate paper key, and preserves the live API specification, which makes it suitable for testing the entire decision-to-order-to-fill software chain. [1]

The broker layer must test:

- authentication failure;
- WebSocket loss;
- market-data staleness;
- REST timeout;
- duplicate client request;
- order reject;
- partial fill;
- cancel/fill race;
- process restart with open orders;
- broker/local position mismatch;
- slow message consumer;
- market open/close boundaries.

Alpaca specifically documents connection limits and slow-client disconnections on its streaming API, so these are not hypothetical edge cases to leave until later. [21]

Run three P&L streams simultaneously:

```
Model-theoretical P&L
          vs
Internal conservative execution P&L
          vs
Broker paper P&L
```

The differences are themselves key diagnostics.

Paper-soak promotion gate

Do not advance merely because total P&L is positive. Require:

- no unexplained order duplication;
- no unresolved position mismatches;
- successful automated recovery from tested disconnections;

- risk kill switch proven during market hours in paper mode;
- complete audit trail for every order;
- strategy decisions reproducible from captured inputs;
- latency distributions within configured bounds;
- no dependence on stale features;
- conservative shadow P&L analyzed separately from broker fills;
- several distinct volatility/regime conditions observed where possible.

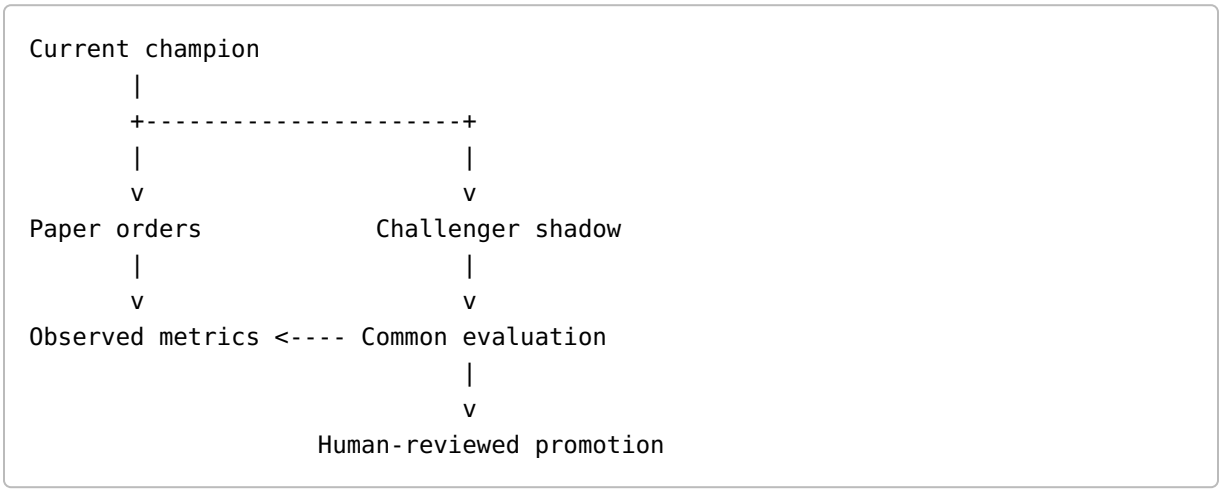
Alpaca cautions that its paper engine differs materially from live trading with respect to impact, latency slippage, queues and other mechanics, so a profitable paper period is evidence of **software and strategy plausibility**, not evidence of live execution profitability. [1]

Monitoring & Iteration

Suggested duration: continuous after integration.

This phase turns a prototype into a governed research platform.

Implement scheduled walk-forward evaluation and champion/challenger comparison:



No model should self-promote based on its own recent reward.

Track drift in:

$$D_{\text{feature}}, \quad D_{\text{regime}}, \quad D_{\text{execution}}, \quad D_{\text{performance}}.$$

Examples include changes in realized volatility, HMM transition probabilities, OU half-lives, spread correlations, trading frequency, slippage, PPO action distributions, and feature missingness.

The ongoing research loop should be:

```
Hypothesis
   ↓
Pre-register experiment config
   ↓
Historical walk-forward
   ↓
Robustness / cost stresses
   ↓
Shadow paper mode
   ↓
Capped paper allocation
   ↓
Post-trade attribution
   ↓
Retain / revise / retire
```

The experiment ledger must retain all variants because multiple-testing bias is otherwise invisible; DSR and backtest-overfitting research directly addresses this problem. [15]

Production acceptance criteria and final blueprint

The project should be considered successfully completed when it meets **systems**, **quantitative**, and **governance** definitions of done—not merely when a chart shows positive simulated returns.

Systems acceptance

The platform must be able to:

1. ingest live data continuously through one controlled adapter;
2. persist and replay all decision-relevant events;
3. produce identical outputs under deterministic historical replay;
4. resume safely after process/network failure;
5. identify and reconcile external broker state;
6. prevent duplicate orders under retry;
7. halt automatically on stale data or state mismatch;
8. explain every rejected, resized, submitted, canceled, and filled order;
9. operate using paper-only credentials;
10. survive injected broker/feed failures without uncontrolled position growth.

Alpaca's own documentation emphasizes that algorithm testing must account for nonfills, network disconnections, retries, and other runtime behavior not visible in a conventional historical backtest. [1]

Quantitative acceptance

A candidate strategy should satisfy all of the following before receiving meaningful paper allocation:

Gate	Required evidence
No leakage	Event-time review and automated tests pass
Benchmark	Compared with simple deterministic baselines
Cost robustness	Positive thesis does not disappear under modest cost perturbation
Parameter robustness	Broad stable region rather than isolated optimum
Walk-forward	Multiple frozen out-of-sample folds
Selection adjustment	Experiment count recorded; DSR/PBO considered
Delay robustness	Results evaluated with signal/execution delay
Regime robustness	Attribution by HMM/volatility state
Tail robustness	Gap/high-volatility/correlation stress tests
Execution robustness	Conservative fill model as well as idealized fills
Model failure	Behavior explicitly defined when HMM/OU/PPO is unavailable

A model should be rejected even with an attractive aggregate Sharpe if almost all returns arise in a single fold, symbol, pair, parameter point, or isolated volatility episode.

Governance acceptance

The final autonomy boundary should be:

```
AI / Quant models MAY:
    estimate regimes
    forecast signals
    rank opportunities
    propose target exposure

AI / Quant models MAY NOT:
    disable risk controls
    modify risk limits at runtime
    switch to live credentials
    alter production code
    approve their own deployment
    conceal missing/stale inputs
    bypass the OMS
    bypass broker reconciliation
```

That boundary is consistent with the broader constrained-RL principle that objectives and safety constraints are distinct objects rather than assuming a reward function will always induce safe behavior. [13]

Recommended final control loop

The production paper agent should ultimately execute this sequence for every decision epoch:

$$\text{Market Event} \rightarrow \text{Validated Feature Snapshot} \rightarrow \text{HMM/OU/Rules/PPO} \rightarrow \text{Target Portfolio} \rightarrow \text{Vol/Kelly Sizing}$$

with one critical feedback condition:

$$\text{Any uncertainty about data, orders, positions, or risk state} \implies \text{less trading, not more trading.}$$

The architecture's most important feature is therefore not PPO, the HMM, or the OU equation. It is the separation between **probabilistic intelligence** and **deterministic authority**. HMM probabilities can be wrong, an OU pair can structurally break, Brownian approximations can miss jumps, and PPO can exploit defects in its training environment. The system remains controllable because none of those components has unilateral authority to transform a prediction into unconstrained broker exposure. Research on mean-reversion nonstationarity, constrained reinforcement learning, and backtest selection bias all reinforce the need for that separation. [22]

For the requested paper-money objective, the strongest implementation path is therefore **data correctness → deterministic quant baselines → robust walk-forward validation → constrained RL challenger → risk-governed OMS → Alpaca paper integration → conservative shadow execution → continuous monitored iteration**. The broker simulator then serves its proper purpose: validating the autonomous trading *system*, while the research framework separately tests whether the underlying quantitative edge survives costs, nonstationarity, multiple testing, and execution assumptions. Alpaca itself makes clear that paper fills omit several mechanisms that distinguish simulation from real trading, which is why maintaining that separation throughout the project is essential. [1]

---

[1] Paper Trading
https://docs.alpaca.markets/docs/paper-trading

[2] IBKR API Home | Interactive Brokers | Documentation
https://ibkrcampus.com/api/getting-started/

[3] [20] Stable-Baselines3 Docs - Reliable Reinforcement Learning ...
https://stable-baselines3.readthedocs.io/?utm_source=chatgpt.com

[4] [14] VectorBT: Getting started
https://vectorbt.dev/?utm_source=chatgpt.com

[5] [17] [21] WebSocket Stream
https://docs.alpaca.markets/docs/streaming-market-data

[6] [11] [19] Databento | Docs
https://databento.com/docs

[7] Common fields, enums and types | Databento standards
https://databento.com/docs/standards-and-conventions/common-fields-enums-types

[8] Predicting Risk-adjusted Returns using an Asset Independent Regime-switching Model
https://arxiv.org/abs/2107.05535?utm_source=chatgpt.com

9 On the Profitability of Optimal Mean Reversion Trading Strategies
https://arxiv.org/abs/1602.05858?utm_source=chatgpt.com

10 Optimal Mean-Reverting Spread Trading: Nonlinear Integral Equation Approach
https://arxiv.org/abs/1701.00875?utm_source=chatgpt.com

12 Policy Networks — Stable Baselines3 2.9.2a0 documentation
https://stable-baselines3.readthedocs.io/en/master/guide/custom_policy.html?utm_source=chatgpt.com

13 Constrained Policy Optimization
https://arxiv.org/abs/1705.10528?utm_source=chatgpt.com

15 The Deflated Sharpe Ratio: Correcting for Selection Bias ...
https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2460551&utm_source=chatgpt.com

16 A New Interpretation of Information Rate - Kelly - 1956
https://onlinelibrary.wiley.com/doi/abs/10.1002/j.1538-7305.1956.tb03809.x?utm_source=chatgpt.com

18 Grafana Alerting | Grafana documentation
https://grafana.com/docs/grafana/latest/alerting/?utm_source=chatgpt.com

22 An Application of the Ornstein-Uhlenbeck Process to Pairs Trading
https://arxiv.org/abs/2412.12458?utm_source=chatgpt.com